

DOCUMENTACIÓN - Entrega 2

Caso 5:	Donaciones "SumarImpacto"
Empresa de Desarrollo:	codeAr
Tecnología principal:	Node.js + Express + MongoDB Atlas/Mongoose + Pug

Integrantes:

1. Borda, Mariana Beatríz
2. Calla Giron, Jairo Natanael
3. Fernández, Analía
4. Sotelo, Cynthia Isabel
5. Suárez, Cristian
6. Virili, Omar Dario

Nombre del documento y de la carpeta drive: DSWB_2A_CodeAr_2C26

ÍNDICE

1. Links **3**

2. Bibliografía **3**

3. Introducción **3**

4. Integrantes y responsabilidades **4**

5. Descripción funcional de la aplicación **5**

- 5.1. Objetivo general 5
- 5.2. Funcionalidades Principales Actuales 6
- 5.3. Reglas de negocio contempladas 6
- 5.4. Requerimientos Funcionales 8
- 5.5. Requerimientos No Funcionales 9

6. Módulos del proyecto **10**

7. Persistencia de Datos **12**

- 7.1. Modelo de Datos 12
- 7.2. Variables de Entorno 13
- 7.3. Identificación de documentos y decisión de migración 13

8. Arquitectura y estructura del código **14**

9. Validación, middlewares y manejo de error **15**

10. Pruebas manuales **17**

11. Criterios de código y buenas prácticas **19**

12. Uso de la inteligencia artificial **20**

1. Links

- **Carpeta de Drive:** https://drive.google.com/drive/folders/1AHoHDZ3tYJ_RuXEE39hcfnf38adIrCRe
- **GitHub:** https://github.com/odv144/codeAr.git
- **Link al Video:** https://youtu.be/c9oYzZOwIb0

2. Bibliografía

Documentos

Material de la cátedra:

- Mapa de contenidos:
  - Bloque 7.1: Introducción a MongoDB.
  - Bloque 7.2: Operadores de elementos y Schema en MongoDB.
- M8- Estructura de documentos: Base de datos Express.
- M9- Creación de una solución: Creación de una API REST completa.

3. Introducción

El presente documento describe la segunda entrega del proyecto de la materia Desarrollo de Sistemas Web Backend desarrollado por el equipo CodeAr para el caso de estudio “SumarImpacto”, una organización que recibe donaciones destinadas a proyectos sociales. El problema planteado consiste en reemplazar el registro manual de donaciones y gastos por una solución que permita administrar la información y mejorar la trazabilidad de los fondos.

En esta instancia se trabajó sobre una aplicación backend desarrollada con Node.js y Express, utilizando ECMAScript Modules (ESM), persistencia mediante MongoDB/Mongoose. También contiene vistas web mediante Pug.

Esta entrega incorpora mejoras respecto del backend inicial: una estructura modular más organizada, conexión con MongoDB Atlas, uso de async/await en operaciones de persistencia, implementación de la autenticación basada en sesiones, middlewares para validación y manejo de errores, validaciones específicas, manejo centralizado de errores, registro de requests.

Objetivos específicos de la instancia

1. Implementar una arquitectura modular y organizada.
2. Integrar una base de datos MongoDB utilizando Mongoose.
3. Utilizar middlewares para validaciones, controles y manejo de errores.
4. Implementar validaciones simples de los datos recibidos.

5. Utilizar programación asíncrona con async/await en las operaciones que interactúan con MongoDB.
6. Documentar correctamente el desarrollo, las responsabilidades, las funcionalidades y las pruebas del proyecto.

Objetivos generales

7. Desarrollar software por encargo a partir de un caso de estudio definido.
8. Integrar al equipo de desarrollo y distribuir responsabilidades por módulos.
9. Aplicar buenas prácticas de programación y organización del código.
10. Avanzar en el desarrollo de un backend de complejidad acotada, con API REST y persistencia.
11. Documentar el desarrollo y preparar una defensa técnica en equipo.

4. Integrantes y responsabilidades

El equipo de desarrollo **codeAr** está compuesto por 6 integrantes con roles especializados:

Integrante	Responsabilidad principal	Trabajo realizado
Omar Virili	Base de datos y módulo Proyectos	• Creación/configuración de la base MongoDB, conexión mediante Mongoose y desarrollo del CRUD del módulo Proyectos. • **Rol**: Backend Lead.
Mariana Borda	Módulo Organizaciones	• Desarrollo del CRUD de Organizaciones, validaciones, baja lógica y conexión del módulo con MongoDB. Documentación. • **Rol:** Desarrollador Backend + QA.
Jairo Calla Giron	Módulo Donantes	• Desarrollo del CRUD de Donantes y su persistencia en MongoDB Atlas. • **Rol:** Desarrollador Backend + QA.

Analía Fernández	Módulo Donaciones	• Desarrollo del CRUD de Donaciones, validaciones básicas y persistencia MongoDB Atlas. • **Rol:** Desarrollador Backend + QA.
Cynthia Sotelo	Módulo Gastos	• Desarrollo del CRUD de Gastos y validación del gasto contra el saldo disponible del proyecto (movimientos de saldo menos gastos acumulados). • **Rol:** Desarrollador Backend + QA.
Cristian Suárez	Plantillas Pug	• Diseño e integración de las vistas Pug, navegación, formularios de alta, detalle, edición y baja. Implementación de la autenticación basada en sesiones. • **Rol:** Desarrollador Frontend/Backend.

5. Descripción funcional de la aplicación

5.1. Objetivo general

SumarImpacto tiene como objetivo administrar la información relacionada con donantes, organizaciones, proyectos, donaciones y gastos, facilitando la consulta y el registro de operaciones vinculadas al financiamiento de proyectos sociales.

La aplicación ofrece una API REST (JSON) para operar sobre los módulos y una interfaz web desarrollada con Pug que permite listar, consultar, crear, modificar y dar de baja registros. El acceso a ambas está protegido por sesión: cualquier usuario autenticado puede consultar, y solo el rol administrador puede crear, modificar o dar de baja.

5.2. Funcionalidades principales actuales

- **Autenticación y roles:** ingreso en `/login` con email o nombre de usuario, cierre de sesión en `/logout`, sesión del servidor con cookie firmada y dos roles (`admin` y `usuario`).
- **Proyectos:** listado de proyectos activos, consulta por ID (solo los activos), alta, modificación y baja lógica. Cada proyecto lleva un historial de movimientos de saldo (`saldo`: monto y fecha) y está asociado a una organización, que debe existir y estar activa al momento del alta .
- **Organizaciones:** listado de organizaciones activas (la API admite `?incluirBajas=true` para incluir las dadas de baja) , consulta por ID (solo las activas), alta, modificación (solo las activas), baja lógica y reactivación mediante `PATCH /organizaciones/:id/reactivar` (solo API). Validaciones específicas de campos, CUIL, teléfono y mail.
- **Donantes:** listado, consulta por ID, alta, modificación y eliminación física. DNI y email se controlan como valores únicos.
- **Donaciones:** listado, consulta por ID, alta, modificación y eliminación física. Cada donación registra monto, CBU, fecha y las referencias a proyecto, donante y organización, que se validan al guardar. El alta y la modificación funcionan desde las vistas web.
- **Gastos:** listado, consulta por ID, alta, modificación y eliminación física. Se valida que el monto del gasto no supere el saldo disponible del proyecto.
- **Vistas Pug:** panel principal, y por cada módulo: listado, detalle, alta, edición y baja con paso de confirmación. Los botones de acción se muestran solo al rol administrador.
- **Manejo de errores:** middleware central que transforma distintos tipos de error en respuestas HTTP consistentes (JSON en la API, vista de `error.pug` en `/vistas`). Algunos controladores de vistas y de los módulos Proyectos y Donantes resuelven sus errores por su cuenta
- **Registro de requests:** middleware que registra método, URL, código de respuesta y duración de cada petición.

5.3. Reglas de negocio contempladas

- Cada donación se registra con un donante (`idDonante`), un proyecto (`idProyecto`) y una organización `(idOrganizacion`); los tres son obligatorios.
- Al guardar una donación se verifica que el proyecto exista y esté activo, que el donante exista y que la organización exista y esté activa.
- Un proyecto puede recibir múltiples donaciones.

- Un gasto se registra asociado a un proyecto activo mediante `idProyecto`.
- **Saldo disponible de un proyecto =** suma de los movimientos del campo `saldo` del proyecto − suma de los gastos del proyecto. El alta y la modificación de gastos validan contra ese saldo (en la modificación se excluye el propio gasto del cálculo).
- Las organizaciones y los proyectos tienen baja lógica (`activa = false` y `fechaBaja`) para conservar el registro y evitar su eliminación física. Un proyecto o una organización dados de baja no se pueden usar en nuevas donaciones o gastos.
- El CUIL de una organización es único, incluso si la otra organización está dada de baja.
- El DNI y el email de un donante son únicos.
- Montos de donaciones y gastos mayores a 0; el CBU debe tener 22 dígitos numéricos.
- Las operaciones de lectura requieren sesión iniciada; las de escritura (alta, modificación, baja) requieren rol administrador.

5.4 Requerimientos Funcionales (RF)

- ***RF-01: Autenticación y Control de Acceso por Roles*** El sistema debe permitir el inicio de sesión mediante nombre de usuario o correo electrónico y contraseña y restringir el acceso garantizando que los usuarios con rol `usuario` solo puedan realizar lecturas, mientras que las operaciones de creación, edición y baja queden reservadas al rol `admin`.
- ***RF-02: Gestión del Módulo de Organizaciones*** El sistema debe registrar, listar, consultar por ID, editar, aplicar baja lógica a organizaciones y reactivar organizaciones. Debe validar la unicidad del CUIL (incluso frente a organizaciones en estado de baja lógica), así como verificar el formato y longitud de campos como teléfono y correo electrónico.

- ***RF-03: Gestión del Módulo de Proyectos*** El sistema debe permitir la creación, consulta general y por ID, modificación y baja lógica de proyectos sociales. Cada proyecto debe estar vinculado obligatoriamente a una organización activa.

- ***RF-04: Gestión del Módulo de Donantes*** El sistema debe permitir dar de alta, consultar, listar, modificar y eliminar físicamente donantes, garantizando que el DNI y el correo electrónico sean únicos mediante índices en la base de datos.

- ***RF-05: Gestión del Módulo de Donaciones*** El sistema debe permitir listar, consultar, registrar, editar y eliminar donaciones. Al registrar una donación, debe verificar la existencia y estado activo del proyecto, donante y organización referenciados, validando un CBU numérico de 22 dígitos y un monto superior a cero.

- ***RF-06: Control e Impedimento de Exceso de Gastos*** El sistema debe permitir crear, listar, editar y eliminar gastos vinculados a proyectos activos. El servidor debe calcular el saldo disponible del proyecto (suma de movimientos de saldo menos gastos acumulados) y rechazar cualquier gasto cuyo monto excede dicho saldo.

- ***RF-07 Manejo de errores*** El sistema debe responder ante errores de validación, identificadores inválidos, registros inexistentes, conflictos por valores duplicados, falta de autenticación, permisos insuficientes y problemas de conexión con la base de datos. Las respuestas deben utilizar códigos HTTP adecuados y mantener un formato consistente con el tipo de solicitud.

- ***RF-08. Persistencia de datos*** El sistema debe almacenar de forma persistente la información de los módulos y los usuarios en MongoDB Atlas, utilizando Mongoose para el modelado y acceso a los datos.

5.5 Requerimientos No Funcionales (RNF)

- **RNF-01. Arquitectura modular** El código debe organizarse en componentes con responsabilidades diferenciadas, separando rutas, controladores, servicios, modelos, middlewares, configuración, utilidades y vistas. Esta estructura debe facilitar el mantenimiento y la incorporación de nuevas funcionalidades.
- **RNF-02. Tecnología y estándares de desarrollo** La aplicación debe desarrollarse utilizando Node.js y Express, con ECMAScript Modules (ESM), y emplear Mongoose como herramienta de acceso a MongoDB. Las operaciones asíncronas de persistencia deben implementarse mediante async/await.
- **RNF-03 Protección de la configuración** Los datos sensibles y parámetros de configuración deben gestionarse mediante variables de entorno, evitando incluir credenciales y secretos directamente en el código fuente. El archivo `.env` debe excluirse del repositorio mediante `.gitignore`.
- **RNF-04 Integridad de los datos** La aplicación debe aplicar validaciones y restricciones de unicidad para evitar registros inconsistentes. También debe verificar las referencias entre entidades y respetar las reglas de negocio relacionadas con los saldos, las bajas lógicas y los valores obligatorios.
- **RNF-05. Mantenibilidad y extensibilidad** La estructura del proyecto debe permitir incorporar nuevos módulos o modificar los existentes sin requerir cambios innecesarios en toda la aplicación. Las validaciones reutilizables y la separación de responsabilidades deben contribuir a reducir la duplicación de código.
- **RNF-06. Usabilidad de la interfaz** La interfaz web debe ofrecer una navegación clara entre los módulos y formularios que permitan ingresar, consultar y modificar los datos. Las operaciones de baja deben incluir una instancia de confirmación para reducir eliminaciones accidentales.
- **RNF-07. Disponibilidad de la persistencia** La aplicación debe establecer la conexión con MongoDB antes de aceptar solicitudes. Si la conexión inicial falla, debe informar el problema y finalizar el proceso. Cuando la base de datos no

esté disponible durante el funcionamiento, los errores deben tratarse de manera controlada.

6. Módulos del proyecto

Módulo de Proyectos

Responsable: Omar | **Persistencia:** MongoDB / Mongoose

- Listar proyectos.
- Consultar proyecto por ID (solo proyectos activos)
- Crear proyecto.
- Editar proyecto.
- Baja lógica del proyecto (`DELETE /proyectos/:id` y vista de eliminación).
- Consultar el detalle desde la vista Pug.

Módulo de Organizaciones

Responsable: Mariana | **Persistencia:** MongoDB / Mongoose

- Listar organizaciones activas (en la API, `?incluirBajas=true` incluye las dadas de baja).
- Consultar por ID (solo activas).
- Crear organización.
- Editar organización (solo si está activa).
- Dar de baja lógicamente.
- Reactivar una organización dada de baja (`PATCH/organizaciones/:id/reactivar`, solo API).
- Validar campos, CUIL, teléfono, email y longitudes.

Módulo de Donantes

Responsable: Jairo | **Persistencia:** MongoDB / Mongoose

- Listar donantes.
- Consultar donante por ID.
- Crear donante.
- Editar donante.
- Eliminar donante (borrado físico).
- DNI y email únicos, garantizados por índice único en la colección.

Módulo de Donaciones

Responsable: Analía | **Persistencia:** MongoDB / Mongoose

- Listar donaciones.
- Consultar donación por ID.
- Crear donación.

- Editar donación.
- Eliminar donación (borrado físico).
- Validar campos obligatorios, monto mayor a 0, CBU de 22 dígitos y existencia de las referencias a proyecto, donante y organización. Estas validaciones se aplican en el servicio y en el esquema de Mongoose; el middleware `validarDonacion` está implementado pero no conectado a las rutas de la API.

Módulo de Gastos

Responsable: Cynthia | **Persistencia:** MongoDB / Mongoose

- Listar gastos.
- Consultar gasto por ID.
- Crear gasto.
- Editar gasto.
- Eliminar gasto (borrado físico).
- Calcular saldo disponible proyecto (movimientos de saldo menos gastos).
- Rechazar gastos que superen el saldo disponible o que apunten a un proyecto inexistente o dado de baja.

Módulo de Plantillas Pug

Responsable: Cristian | **Persistencia:** Vistas del servidor

- Panel principal y navegación entre módulos.
- Listados por módulo , con nombres legibles de proyecto, donante y organización.
- Formularios de alta y edición, con selectores para las referencias.
- Detalle de organizaciones, proyectos, donantes, donaciones y gastos.
- Baja de los cinco módulos con paso de confirmación (baja lógica en organizaciones y proyectos; borrado físico en donantes, donaciones y gastos).
- Pantalla de ingreso (login) y vista de error para las respuestas provenientes de `/vistas`.
- Botones de acción visibles solo para el rol administrador.

Módulo de Autenticación y control de acceso

Responsable: Cristian | **Persistencia:** MongoDB (colección `users`) / sesión del servidor

- Ingreso con email o nombre de usuario y contraseña; contraseñas almacenadas con hash (bcryptjs).
- Sesión con express-session y cookie `sumarimpacto.sid`.
- Roles `admin` y `usuario`; autorización aplicada en el servidor (`isAuthenticated` e `isAdmin`).
- Usuarios de desarrollo creados de forma idempotente por `scripts/seedUsers.js`, que además sincroniza contraseña y rol cuando difieren de lo declarado en el entorno.

Evolución prevista

- Refactorizar y modificar el código para incorporar un esquema de seguridad más robusto y desacoplado. Esta mejora contempla:
  - ***Integración de Passport.js:*** Para estandarizar y modularizar las estrategias de autenticación en el servidor Node.js y Express.
  - ***Implementación de JSON Web Tokens (JWT):*** Para gestionar sesiones sin estado (stateless) y dotar de mayor seguridad a las APIs del sistema.
  - ***Gestión avanzada de cookies y seguridad de credenciales:*** Aplicando prácticas recomendadas para el almacenamiento seguro, hashing de contraseñas y un control granular de permisos según los roles de usuario en MongoDB.
- Migrar los identificadores a _id/ObjectId

7. Persistencia de datos

El proyecto utiliza MongoDB Atlas como base de datos NoSQL persistente, integrado mediante Mongoose ODM en Node.js. La conexión se centraliza en `config/db.js` mediante `mongoose.connect()`, usando las variables de entorno `MONGO_URI` y `DBNAME`. El servidor se conecta antes de aceptar tráfico y, si no logra conectar, informa el error y finaliza el proceso.

7.1. Modelo de datos

Colección (modelo)	Identificador	Campos principales	Baja
`proyectos` (Proyectos)	`idProyecto`	`idOrganizacion`, `nomProyecto`, `descripcion`, `saldo` (lista de {monto, fecha}), `activa`, `fechaBaja`	Lógica
`organizaciones` (Organizacion)	`idOrganizacion`	`nombre`, `tipo`, `cuil` (único, formato XX-XXXXXXXX-X), `telefono`, `mail`, `direccion`, `responsable`, `activa`, `fechaBaja`	Lógica
`donantes` (Donante)	`idDonante`	`nombre`, `apellido`, `dni` (único), `telefono`, `email` (único), `monto`, `fecha`	Física
`donaciones` (Donacion)	`idDonacion`	`monto` (> 0), `cbu` (22 dígitos), `fecha`, `idProyecto`, `idDonante`, `idOrganizacion` (todos obligatorios)	Física

Colección (modelo)	Identificador	Campos principales	Baja
`gastos` (Gasto)	`idGasto`	`idProyecto`, `descripcion` (3 a 200 caracteres), `monto` (> 0), `fecha`	Física
`users` (User)	`userName` y `email` (únicos)	`password` (hash bcrypt, no se devuelve por defecto), `firstName`, `lastName`, `role` (`admin` o `usuario`), `createdAt/updatedAt`	—

7.2. Variables de entorno

Los datos sensibles no se incorporan al código fuente: la aplicación los toma de un archivo .env (excluido del repositorio mediante .gitignore).

Variable	Requerida	Uso
`MONGO_URI`	Sí	Cadena de conexión a MongoDB Atlas.
`DBNAME`	Sí	Nombre de la base de datos.
`PORT`	No	Puerto del servidor.
`SESSION_SECRET`	En producción	Secreto para firmar la cookie de sesión. Sin él, en producción la aplicación no arranca; en desarrollo se genera uno efímero con una advertencia.
`SESSION_TTL_MINUTES`	No	Duración de la sesión en minutos (por defecto 480).
`NODE_ENV`	No	Con el valor `production` la cookie de sesión se marca como `secure`.
`ADMIN_USER`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`	No	Credenciales del usuario `administrador` de desarrollo que crea el seed.
`OPERADOR_USER`, `OPERADOR_EMAIL`, `OPERADOR_PASSWORD`	No	Credenciales del usuario con rol `usuario` que crea el seed.

Si faltan las variables de usuarios, `scripts/seedUsers.js` utiliza valores por defecto de desarrollo; no deben usarse en producción.

7.3. Identificación de documentos y decisión de migración

En esta entrega cada entidad se identifica con una clave de negocio numérica propia (`idProyecto`, `idOrganizacion`, `idDonante`, `idDonacion`, `idGasto`), que se asigna al crear el registro como el último valor existente más uno. El parámetro `:id`

de las rutas corresponde a esa clave y no al `_id` de MongoDB. Los modelos de Organizaciones, Donaciones y Gastos ocultan el `_id` en la respuesta JSON; los de Donantes y Proyectos todavía lo exponen.

Esta decisión responde al proceso de migración progresiva desde la implementación inicial basada en archivos JSON hacia MongoDB/Mongoose: mantener claves numéricas permitió conservar la compatibilidad con las rutas existentes mientras se incorporaba la nueva capa de persistencia, evitando modificar simultáneamente todos los módulos.

Se reconoce que el mecanismo nativo de identificación de MongoDB es el campo `_id`, generalmente representado mediante ObjectId. Por lo tanto, el uso actual de claves numéricas se considera una decisión transitoria de compatibilidad y no el modelo definitivo.

Para la Entrega 3 se prevé completar esta adaptación, reemplazando progresivamente las claves numéricas por `_id`/ObjectId en los modelos, controladores, rutas, validaciones, consultas y pruebas correspondientes.

8. Arquitectura y estructura del código

La aplicación está organizada por responsabilidades, separando rutas, controladores, servicios, modelos, middlewares, utilidades, configuración y vistas. Esta organización favorece la mantenibilidad y permite que cada integrante trabaje sobre un módulo sin concentrar toda la lógica en un único archivo.

Carpeta / archivo	Responsabilidad
index.js	Punto de entrada; configura Express, sesiones, middlewares, rutas, Pug, conexión a MongoDB, seed de usuarios y arranque del servidor.
config/	Configuración de la conexión a MongoDB.
routes/	Definición de endpoints REST y rutas de las vistas y rutas de autenticación.
controllers/	Recepción de requests, ejecución de lógica de aplicación y generación de respuestas. (API/vistas).
services/	Lógica de persistencia y operaciones de negocio de módulos MongoDB.
models/	Esquema de Mongoose: Organizaciones, Proyectos, Donantes, Donaciones, Gastos y User.

middlewares/	Autenticación y autorización, validación de body e ID, validación de organizaciones, logging y manejo centralizado de errores.
utils/	Errores de aplicación (`AppError`) y funciones de validación reutilizables.
views/	Plantillas Pug: layout, login, panel, listados, formularios, detalle, modificación, eliminación , baja lógica y errores.
public/	Hoja de estilos, logo y favicon.
scripts/	`seedUsers.js`: creación idempotente de los usuarios de desarrollo (`npm run seed` o automática al arrancar).

9. Validaciones, middlewares y manejo de errores

Middlewares implementados

Middleware	Función
`requestLogger`	Registra método HTTP, URL, código de respuesta y duración de la petición.
`isAuthenticated`	Exige sesión activa: la API responde 401 y las vistas redirigen a `/login`.
`isAdmin`	Exige rol `admin`; responde 403 si no lo tiene.
`validarBody`	Verifica que POST/PUT reciban un objeto JSON con contenido. Aplicado en Organizaciones y Gastos.
`validarId`	Controla que el parámetro `:id` sea un entero positivo. Aplicado en Organizaciones, Gastos y en las vistas; no se aplica en las rutas de API de Proyectos, Donantes ni Donaciones.
`validarOrganizacion`	Valida tipos, longitudes y formatos de los datos de una organización (CUIL, teléfono, mail). Deja los datos limpios en `req.datosOrganizacion`.
`validarGasto`	Valida `idProyecto`, descripción (3 a 200 caracteres), monto (> 0) y fecha. Deja los datos en `req.datosGasto`.
`validarDonacion`	Valida monto, CBU, fecha y referencias numéricas de una donación. Está implementado pero no conectado a las rutas (ver 13).

Middleware	Función
`validarDonacion`	Valida monto, CBU, fecha y referencias numéricas de una donación. Está implementado pero no conectado a las rutas, por lo que `req.datosDonacion` nunca se completa.
`notFound`	Genera un error 404 cuando ninguna ruta coincide.
`errorHandler`	Centraliza las respuestas de error y traduce errores de la aplicación (`AppError`), validación de Mongoose, valores inválidos, claves duplicadas, falta de conexión, JSON mal formado y errores internos.

Manejo de errores HTTP

Formato de la API: `{ mensaje, errores? }`, donde `errores` es la lista de detalles de validación cuando corresponde. En `/vistas` el mismo error se muestra en la plantilla `error.pug`.

Situación	Código	Respuesta prevista
Body vacío o no es un objeto JSON	400	El cuerpo de la petición está vacío o no es un objeto JSON (indica enviar `Content-Type: application/json`).
JSON mal formado	400	El cuerpo de la petición no es un JSON válido.
ID inválido	400	El id debe ser un número entero positivo.
Dato de organización o gasto inválido	400	Datos de la organización (o del gasto) inválidos, con la lista de errores.
Error de esquema Mongoose / valor inválido	400	Datos inválidos (con detalle por campo) o Valor inválido para '<campo>'.
Referencia inexistente o dada de baja	400	El proyecto indicado no existe o está dado de baja / El donante indicado no existe / La organización indicada no existe o está dada de baja.
Gasto superior al saldo	400	El monto del gasto supera el saldo disponible del proyecto.
No autenticado	401	No autenticado (API) o redirección a `/login` (vistas). En el login: `{ error: "Credenciales inválidas" }`.
Sin permisos de administrador	403	Acceso denegado: requiere permisos de Administrador.
Registro inexistente	404	Mensaje específico: Organización / Gasto / Donación no encontrada.
Ruta inexistente	404	Ruta no encontrada: <MÉTODO> <URL>.
CUIL duplicado	409	Ya existe una organización con ese CUIL (distingue si la existente está dada de baja).

Situación	Código	Respuesta prevista
Valor único duplicado (clave 11000)	409	Ya existe un registro con ese <campo> (<valor>).
MongoDB no disponible	503	La base de datos no está disponible. Intente nuevamente en unos instantes.
Error inesperado	500	Error interno del servidor; el detalle técnico queda en la consola.

Aclaración: los controladores de Donantes y Proyectos resuelven sus errores por su cuenta y no pasan por el `errorHandler`, responden `{ mensaje }` o `{ mensaje, error }`. Por lo que algunos códigos de la tabla no aplican a esos módulos.

10. Pruebas manuales

Precondición: todas las pruebas de API requieren una sesión iniciada (cookie `sumarimpacto.sid`); las de escritura requieren rol administrador.

ID	Endpoint	Caso	Resultado esperado	Verificación
A-01	`POST /login`	Credenciales válidas	200 y `redirectUrl`	Coincide
A-02	`POST /login`	Contraseña incorrecta	401 Credenciales inválidas	Coincide
A-03	`GET /proyectos`	Sin sesión	401	Coincide
A-04	`POST /proyectos`	Sesión con rol usuario	403	Coincide
A-05	`GET /vistas`	Sin sesión	Redirección a /login	Coincide
P-01	`GET /proyectos`	Consultar listado	200 y lista de proyectos activos	Coincide
P-02	`GET /proyectos/:id`	Consultar proyecto existente	200 y proyecto	Coincide
P-03	`POST /proyectos`	Crear con campos obligatorios	201 y proyecto creado	Coincide
P-04	`PUT /proyectos/:id`	Modificar proyecto	200 y proyecto actualizado	Coincide

ID	Endpoint	Caso	Resultado esperado	Verificación
P-05	`DELETE /proyectos/:id`	Baja de proyecto existente	200; activa=false	Coincide
O-01	`POST /organizaciones`	Crear organización válida	201	Coincide
O-02	`POST /organizaciones`	Enviar body vacío	400	Coincide
O-03	`POST /organizaciones`	CUIL duplicado	409	Coincide
O-04	`DELETE /organizaciones/:id`	Baja lógica	200; activa=false	Coincide
O-05	`GET /organizaciones/abc`	ID no numérico	400	Coincide
D-01	`GET /donaciones`	Consultar el listado de donaciones con sesión iniciada	200 y lista de donaciones	Coincide
D-02	`GET /donaciones/:id`	Consultar una donación existente	200 y datos de la donación	Coincide
D-03	`POST /vistas/donaciones`	Alta desde el formulario web con datos válidos.	Redirección a /vistas/donaciones	Coincide
DNT-01	`GET /donantes`	Consultar el listado de donaciones con sesión iniciada	200 y el listado de donantes registrados.	Coincide
DNT-02	`POST /donantes`	Crear donante válido	201 y el donante se registra correctamente	Coincide
G-01	`POST /gastos`	Gasto dentro del saldo	201	Coincide
G-02	`POST /gastos`	Gasto superior al saldo	400	Coincide

ID	Endpoint	Caso	Resultado esperado	Verificación
G-03	`PUT /gastos/:id`	Modificar excediendo saldo	400	Coincide
G-04	`POST /gastos`	Proyecto inexistente o dado de baja	400	Coincide
E-01	`GET /ruta-inexistente`	Ruta no definida	404	Coincide

11. Criterios de código y buenas prácticas

Estructura y buenas prácticas

- Uso de ECMAScript Modules mediante `type=module en package.json`.
- Separación de rutas, controladores, servicios, modelos y vistas.
- Capa de servicios para las operaciones principales sobre MongoDB.
- Modelado de datos con Mongoose, con validaciones de esquema (requeridos, rangos, formatos e índices únicos).
- Uso de async/await en operaciones asíncronas con MongoDB.
- Variables de entorno para la configuración sensible (conexión, secreto de sesión y credenciales de desarrollo).
- Manejo centralizado de errores mediante `AppError` y `errorHandler` (aplicado en Organizaciones, Gastos, Donaciones y vistas).
- Logging de requests mediante requestLogger.
- Validaciones reutilizadas entre la API y los formularios web en el módulo Organizaciones (`validarDatosOrganizacion`). En Gastos y Donaciones los formularios web se apoyan en las validaciones del esquema de Mongoose y de los servicios.
- Baja lógica en Organizaciones y Proyectos para conservar el historial.
- Comentarios en el código para mejorar la comprensión.

Seguridad

- Contraseñas almacenadas con hash (bcryptjs) y nunca devueltas por las consultas.
- Cookie de sesión `HttpOnly`, `SameSite=Lax` y `secure` en producción; regeneración del id de sesión al ingresar.

- Autorización aplicada en el servidor mediante middlewares, no solo ocultando botones en las vistas.
- Mensaje único ante credenciales inválidas, para no revelar si el usuario existe.
- Seed de usuarios idempotente: no duplica registros ni falla al reiniciar con nodemon.

12. Uso de inteligencia artificial

Durante la elaboración del proyecto y su documentación se utilizó inteligencia artificial generativa como herramienta de apoyo. Su utilización se orientó a tareas de asistencia y revisión, manteniendo el control del equipo sobre el código y las decisiones técnicas.

- **Documentación:** apoyo para organizar la estructura del documento, mejorar redacción técnica y convertir la información del proyecto en una presentación formal.
- **Revisión:** apoyo para detectar inconsistencias, revisar nombres, endpoints, validaciones y proponer casos de prueba.
- **Aprendizaje:** consulta de conceptos relacionados con Express, MongoDB/Mongoose, middlewares, async/await y Pug.